

Forward Deployed AI Engineer Roadmap 2026

Forward Deployed Engineer (a.k.a. FDE) is becoming a hot job in the market. One can become FDE in different areas such as AI, Data Engineering etc. This roadmap is for AI Engineering FDEs.

Watch [this video](#) first to understand what exactly this role is, what the skills required for it are etc.

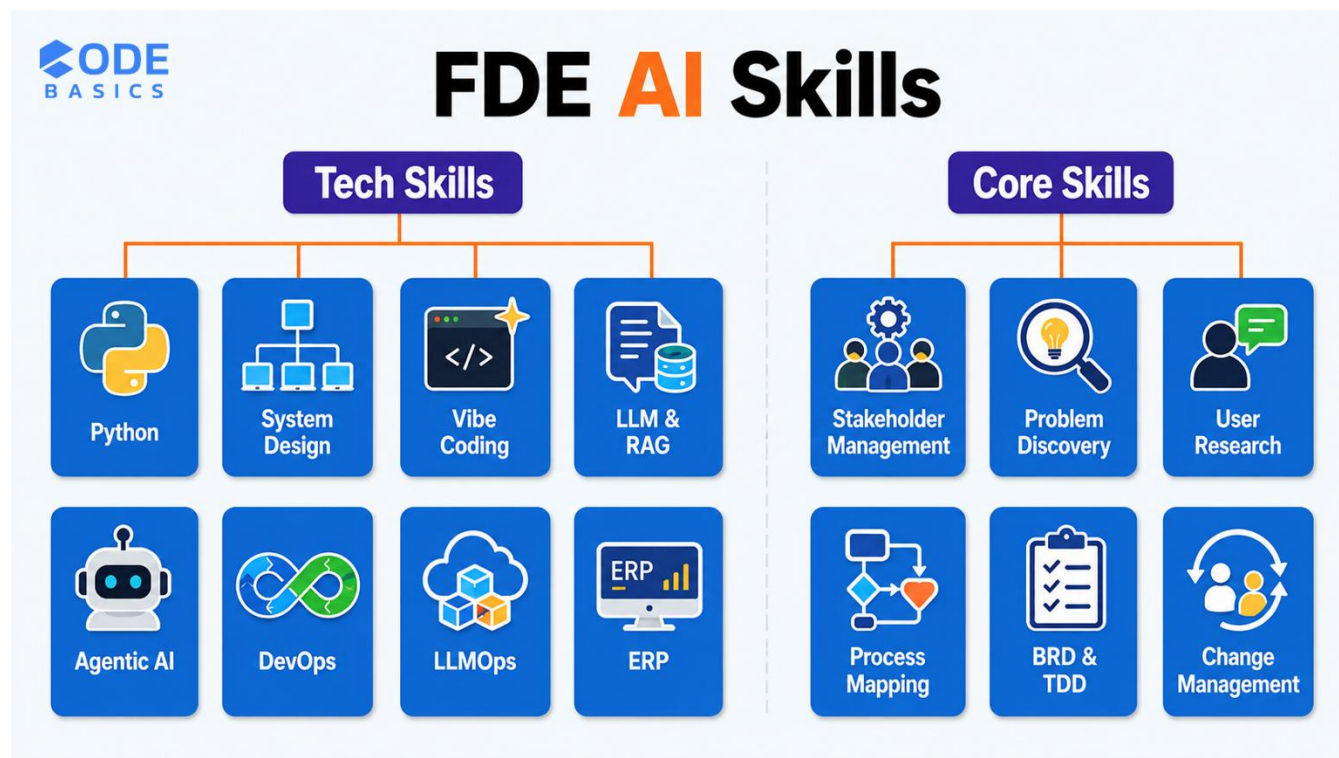
How was this Roadmap built?

We built this roadmap by interviewing these FDE's who work in different domains:

1. Pankaj Jaiswal: <https://www.linkedin.com/in/pankaj29/>
2. Rushi Gandhi: <https://www.linkedin.com/in/rushi0508/>
3. Pranav Modh: <https://www.linkedin.com/in/modhpranav/>
4. Daksh Trehan: <https://www.linkedin.com/in/dakshtrehan/>

We also analyzed FDE jobs and prepared a list of skills mentioned in FDE job postings: Check this [code](#)

Based on our research, here are the tool skills and soft skills required for this role.



Who can follow this roadmap? Pre-requisites

Any person with minimum one year of programming experience (in any programming language) can follow this. You should have following fundamentals clear to follow this roadmap.

1. **Programming fundamentals** — variables, data types, loops, conditionals, functions, error handling, and basic OOP (classes and objects) in any language
2. **SQL fundamentals** — SELECT, WHERE, JOIN, GROUP BY, and reading a simple schema. As an FDE you will constantly pull and inspect enterprise data
3. **Git & GitHub** — clone, commit, push, branching, and raising a pull request
4. **How the web works** — what an HTTP request/response is, what JSON looks like, and what an API does at a high level (you'll go deep on APIs inside the roadmap)
5. **Debugging habit** — ability to read an error message, print/log your way to the problem, and search documentation instead of giving up

Roadmap Overview

FDE = AI engineer + a slice of AI PM

In Phase 1, you will learn core AI engineering skills. In Phase 2, you will learn product management and other soft skills required to become a successful FDE.

Weeks	Focus
Phase 1	Technical Skills
1	Python for Enterprise AI — FastAPI, Pydantic, Async
2	Vibe Coding
3	LLM Fundamentals and RAG
4–6	Agentic AI, Multi-Agent Systems, MCP
7	ERP & Enterprise Data Integration
8	DevOps for AI — Docker, Kubernetes, CI/CD
9–10	LLMOps — Observability, Evaluation & Security
11–12	System Design for Enterprise AI — HLD & LLD
Phase 2	Product Management & Soft Skills
13–14	Problem Discovery & User Research
15	AS-IS Process Mapping with Swim-lane Diagrams
16–17	Business Requirement Document (BRD) Mastery
18	Technical Design Document (TDD)
19–20	Stakeholder Management & Communication
21–22	UAT & Go-Live
23–24	Change Management, Handover & Proving ROI

Phase 1: Technical Skills (Week 1–12)

Build and ship enterprise AI: Python, RAG, agents, ERP data, DevOps, LLMOps, and system design.

Week 1, 2: Python for Enterprise AI — FastAPI, Pydantic, Async 🦄

Python is the Programming language for AI. Python is a MUST in any FDE job posting. You will start with basics and go advanced in how python is used for enterprise AI

Topics

- Python basics: for loop, data types, variables, if else
- FastAPI: route definitions, request/response models with Pydantic, background tasks, middleware
- Pydantic for data validation: schema definition, custom validators, nested models
- Error handling patterns: HTTPException, custom exception handlers, structured error responses
- Environment management: .env files, secrets management, never hardcoding API keys
- Calling LLMs from Python (using groqcloud etc.)

Learning Resources

- Python for AI (YouTube) – [Link](#)
- FastAPI full tutorial playlist (YouTube) — [Link](#)
- Pydantic v2 official docs — [Link](#)
- Python asyncio - [Link](#)

Assignments

- ☐ Build a FastAPI service for a work-order tracker with 4 endpoints (create work order, list all, update status, get by ID) using Pydantic models for request/response validation and proper error handling (HTTPException, structured error responses)
- ☐ Add a /summarize endpoint that calls an LLM (GroqCloud free tier) to summarize a work order description - API key loaded from .env, never hardcoded
- ☐ Push the project to GitHub with a README explaining how to run it

Week 3: Vibe Coding 🎵

*An FDE who codes with AI is 3x faster. An FDE who blindly trusts AI code fails in production. This week teaches you the difference. During my conversation with MakeMyTrip's CTO (Sambit Sarangi), he told me that the thing they check during interviews is **how well you can review AI code**. Hence using AI to code faster is good but you should also know fundamentals so that you can review and validate AI generated code.*



Topics

- As an FDE, you are expected to build things fast with AI, but you are responsible for the code you ship not the AI. AI can hallucinate and make mistakes, as a responsible FDE, you must build a strong testing layer to make sure things do not break in production.
- Companies use platforms such as <https://www.aikido.dev/> to enhance security of things built by developers and AI
- Tools: Claude Code, Cursor, GitHub Copilot, Antigravity — pick one, go deep
- Claude Skills, Claude.md, Context management
- Spec-first + planner workflow: SDD → implementation plan → execute step by step. Say this to Cursor: "Ask as many questions as possible"
- Validating AI-generated code: read every diff, catch hallucinated APIs, weak error handling, hardcoded secrets — this is now an FDE interview skill

Learning Resources

- Claude tutorial (YouTube) - [Link](#)
- Antigravity tutorial (YouTube) - [Link](#)
- Cursor tutorial (YouTube) - [Link](#)

Assignments

- ☐ Extend your Week 1,2 work-order tracker using Claude Code or Cursor: first write a CLAUDE.md with 5 project conventions (naming, error handling, folder structure, type hints, no hardcoded secrets), then add 2 new features through AI — technician assignment and a priority field with validation
- ☐ Code review drill: ask AI to generate ~100 lines of code for a new feature, then find and fix at least 3 issues in it (hallucinated API, missed edge case, weak error handling) without AI help

Week 4, 5: LLM Fundamentals and RAG

In our company [AtliQ Technologies](#), 30 to 40% projects require some type of RAG (Retrieval Augmented Generation) and this is a pattern throughout the industry. Before you start learning RAG, you need to have LLM fundamentals clear.



Topics

- Understanding AI Landscape: ML, DL, Gen AI, Agentic AI
- Transformer architecture and how LLMs work.
- LLM core concepts: temperature, top-p, top-k, context window, autoregressive generation
- RAG, chunking strategies, vector RAG and vector less RAG
- Ingestion pipeline: PDFs, Word SOPs, Excel exports → chunking (fixed, recursive, semantic) → embeddings
- Vector databases: You need to at least one vector db (e.g. ChromaDB, Qdrant, PineCone, PgVector)
- Retrieval quality: semantic vs keyword (BM25) vs hybrid search, re-ranking
- Hallucination guardrails: grounding, citations, "I don't know" behavior
- Have cost and latency in mind all the time. You need to experiment with different models (both open-source and commercial), and pick the one which is most cost effective and meets latency needs
- Using open-source models (such as the ones from <https://huggingface.co/models>) for certain parts of projects is often a good idea to keep costs low

Learning Resources

- AI Basics (Explains entire AI Landscape: ML, DL, Gen AI etc.) – [YouTube Tutorial](#)
- What is LLM – [YouTube Tutorial](#)
- LLM Explained Briefly – [YouTube Tutorial](#)
- Chromadb getting started - [Link](#)
- RAG Basics – [YouTube Tutorial](#)
- Transformer explanation – [YouTube Tutorial](#)

Assignments

- ☐ Build a RAG system over 10 real PDFs (equipment manuals, HR policies, or vendor contracts): ingest → chunk → embed → answer questions with citations to the source page
- ☐ Write a Kaggle notebook or GitHub repo for your RAG project and share on LinkedIn

Week 6, 7, 8: Agentic AI, Multiagent Systems, MCP

The world is obsessed with Agentic AI. As an FDE, you will build agentic automations while working at the client site.

Topics

- What exactly is an agent? ReAct loop
- Tool calling: defining tools with clear schemas and boundaries, structured JSON output
- Build agent using Langchain
- Build a shopping agent using Langchain
- Multiagent systems using Langgraph
- LangGraph: state, nodes, conditional edges, memory persistence
- Multimodal agents
- Human-in-the-loop: interrupt and route to a person when confidence is low or the action is irreversible
- Guardrails and eval us
- MCP (Model Context Protocol): the standard way to expose enterprise tools to agents
- Guardrails and evals: limiting what an agent can do, testing it before a client does

Learning Resources

- Langchain academy free courses - [Link](#)
- Agentic AI crash course (includes two projects) - [YouTube — Codebasics](#)
- Deeplearning AI agentic ai course - [Link](#)
- Langgraph tutorial – [YouTube - Codebasics](#)
- What is MCP? – [YouTube - Codebasics](#)
- Build your first MCP server – [Youtube – Codebasics](#)

Assignments

- ☐ Build a work-order triage agent on top of your Week 1 tracker: it reads an incoming complaint, uses your RAG system to check the equipment manual, decides priority, and creates the work order via your API — all through tool calls
- ☐ Add a human-in-the-loop interrupt: if the agent's confidence is below 85% or priority is P1, route to a "needs review" state instead of auto-creating
- ☐ Record a 5-minute demo video explaining the agent's decision flow as if presenting to a client's IT team, and post it on LinkedIn
- ☐ Build a 3-agent LangGraph system: Agent 1 extracts key fields from RFQ/vendor documents, Agent 2 validates completeness against approved vendor list, Agent 3 drafts a procurement recommendation and posts to a mock approval API

FDE Development Philosophy: You are not building a demo. You are building production code that will run inside a client's environment with their messy legacy data, their IT security policies, and their end-users who are not technical. Build defensively. Document everything.

Week 9: ERP & Enterprise Data Integration

You will almost always build AI on top of an ERP — SAP, NetSuite, Dynamics, Tally, or a custom one. Learn ERP literacy and how to pull data out. Go deeper only if your client lives inside SAP (see the Bonus section at the end of this roadmap).

Topics

- Many FDE job openings have this job title “FDE / **Integration Engineer**”. You are required to build an integration layer on top of ERP that can interact with ERP data and make things ready for AI layer. There are in fact SAAS companies such as <https://www.refold.ai/> that builds integration layer as a product. If your client is not using such SAAS products then you as an FDE will be responsible for building this layer.
- ERP landscape: SAP, Oracle NetSuite, MS Dynamics, Tally — what a system of record is, master data vs transactional data
- SAP module map at a glance (PM, CS, MM, SD, HR): know what each module owns — nothing deeper
- How data gets out: pre-built connectors (Palantir Foundry, Databricks, Airbyte), OData/REST APIs, MCP servers
- SAP OData in practice: consuming S/4HANA APIs (Equipment Master, Work Orders) via the free sandbox — no SAP installation needed
- REST API design for enterprise: authentication (OAuth2, API keys), rate limiting, retry logic, idempotency
- Data quality realities: duplicate masters, cryptic codes needing lookup tables, stale data
- The scarce skill: knowing what to ask the client’s IT team for — data dictionary, API catalogue, authorizations, sandbox access

Learning Resources

- SAP API Business Hub — live sandbox testing (free) - [Link](#)
- SAP Developers — OData & API integration tutorials (free) - [Link](#)
- REST API crash course (YouTube — Traversy Media) - [Link](#)

Assignments

- ☐ Call the Equipment Master OData API (API_EQUIPMENT) from the **SAP Business Accelerator Hub's free public sandbox** (api.sap.com — create a free account, click “Show API Key” on the API's page, and call the sandbox URL with that key; **no SAP system or license needed**) and retrieve equipment details including functional location and maintenance plant
- ☐ Build a Python FastAPI endpoint that wraps this call and returns structured JSON (equipment status, last maintenance date, assigned technician) — then connect it as a tool to your agent
- ☐ Write a one-page integration spec (source system, API, auth, fields, refresh frequency) as if submitting to a client’s IT team

Week 10, 11: DevOps for AI — Docker, Kubernetes, CI/CD

FDE is responsible for code deployment and continuous delivery. For this knowledge of Dev Ops is essential

Topics

- As an FDE, you will be responsible for building the infrastructure behind the AI layer. A disciplined approach and reusable documentation will help everyone save time and avoid frustration.
- Docker: multi-stage builds, container registries (AWS ECR, Azure ACR), optimizing image size for AI apps
- Kubernetes: deployments, services, config maps, secrets, resource limits, liveness/readiness probes
- CI/CD with GitHub Actions: lint → test → build → push → deploy pipeline for AI applications
- API Gateway patterns: rate limiting, authentication (OAuth2/JWT), CORS, request routing
- Infrastructure as Code: Terraform basics for provisioning cloud resources
- Secrets management: AWS Secrets Manager, Azure Key Vault — never in environment variables
- A/B testing and canary releases for AI model versions
- On-premise Kubernetes: kubeadm, Rancher, air-gapped Docker registries, private LLM hosting (Ollama, vLLM)

Learning Resources

- Docker full course (YouTube — TechWorld with Nana) - [Link](#)
- Kubernetes crash course (YouTube — TechWorld with Nana) - [Link](#)
- GitHub Actions CI/CD full course (YouTube) - [Link](#)
- Terraform for beginners (YouTube — freeCodeCamp) - [Link](#)

Assignments

- ☐ Containerize your Week 4–6 multi-agent system with Docker and push to Docker Hub or AWS ECR
- ☐ Write a GitHub Actions workflow that runs tests and deploys on push to main
- ☐ Deploy to a cloud VM or cloud container service (AWS ECS, Azure Container Apps) and share the live URL

Week 12, 13: LLMOps — Observability, Evaluation & Security

Deploying your AI system is step one. LLMOps is keeping it observable, evaluated, secure, and within budget once real users depend on it.

Topics

- LLM trace logging: capturing inputs, outputs, latency, and token usage per request (LangSmith, LangFuse, OpenTelemetry)
- Metrics dashboards with Prometheus + Grafana: requests/min, P95 latency, error rate, token cost per day
- Evaluation in production: regression test suites, hallucination rate monitoring, comparison against ground truth
- Cost management: token budgets, model routing (lightweight models for simple tasks, premium models only where accuracy demands), alert thresholds
- Security: OWASP Top 10 for LLMs — prompt injection, data leakage, insecure output handling
- RBAC and SSO: permission layers for multi-tenant AI apps, OAuth2 / SAML / Azure AD integration patterns
- Scaling and reliability: horizontal pod autoscaling, load testing with Locust, defining SLAs

Learning Resources

- LangSmith observability — official docs (free) - [Link](#)
- LangFuse open-source LLM Ops — docs - [Link](#)
- LLM evaluation with RAGAS — docs - [Link](#)
- OWASP Top 10 for LLMs - [Link](#)
- What is MLOps? (YouTube) - [Link](#)
- MLFlow tutorial (YouTube) - [Link](#)

Assignments

- ☐ Add LangSmith or LangFuse tracing to your AI agent and capture 50 test traces across different work order types
- ☐ Set up a Grafana dashboard with: requests/min, P95 latency, error rate, token cost/day
- ☐ Write a security threat model for your AI system: identify 3 prompt injection risks and document mitigations
- ☐ Load test your API with 50 concurrent users using Locust and document the results

Week 14, 15, 16: System Design for Enterprise AI — HLD & LLD

Calling an LLM API is easy. Building a system around it that is fast, cheap, reliable, and doesn't embarrass you in production — that's AI System Design. FDE interviews open with system design. HLD proves you can architect the system; LLD proves you can hand a precise spec to a developer — or to an AI coding agent. Read the sample SDD in the Document Pack alongside this week: it is the same field-service system you are designing, written the way a client's architecture board expects to receive it.

Topics

- As an FDE, you need to have understanding of client's current system design and suggest enhancement for integration layer that you are building such that it does not impact their current usage.
- Why AI system design is different: an LLM is non-deterministic, expensive per call, slow, and confidently wrong. Almost every unusual box in an AI architecture exists to defend against one of those four properties — be able to name which
- The model layer: API models vs self-hosted, and model routing. A small model for extraction and classification, a frontier model only where synthesis is genuinely required. This is the single largest cost lever you control
- The retrieval layer: ingestion path vs query path, hybrid search (vector + keyword), reranking, chunking strategy, metadata filtering. When RAG answers are bad it is retrieval 8 times out of 10 — debug there before blaming the model
- Orchestration: tool calling, the think-act-observe loop, and the reliability maths — 95% per step across 10 steps lands near 60% end to end. Prefer fixed workflows for known enterprise processes; reserve open-ended agents for genuinely open-ended tasks
- Latency and cost design: time-to-first-token vs total time, streaming, and the caching ladder — exact-response cache, then semantic cache, then stable prompt prefix. Plus output caps and prompt hygiene. Cost is a designed property, not an invoice you discover at month end
- The trust layer: input and output guardrails, evals as the unit tests of an AI system (no evals, no production), and end-to-end tracing so you can tell whether retrieval or generation failed
- Degradation and fallback design: primary model → secondary → graceful manual path, with every tier announced to the user rather than failing silently. Hope is not an architecture
- HLD (High-Level Design): components and how they interact, technology stack with rationale and the trade-off accepted, deployment topology (cloud vs on-premise, ERP connection), and NFRs — enterprise AI needs reliability, data residency and cost control, not millions-of-users scale

- LLD (Low-Level Design): API endpoint specs with OpenAPI/Swagger, request/response contracts, database schema (entities, relationships), and sequence diagrams — including the failure sequence, not only the happy path
- LLD for AI systems: agent and tool boundaries, a deterministic gate in front of every irreversible action, model and prompt configuration, and drawing the line between what is configuration and what is code
- Architecture Decision Records: the decision, the alternatives you rejected, and what it costs you.
Diagramming tools: draw.io (free), Eraser.io (AI-native diagram-as-code), Mermaid — keep the source, not just the picture

Learning Resources

- HLD System design playlist (YouTube — Gaurav Sen) - [Link](#)
- AI System design (Youtube – Codebasics) – Tutorial coming up soon
- OpenAPI / Swagger specification - [Link](#)
- draw.io for architecture diagrams (free) - [Link](#)
- Eraser.io — AI-native DSL diagrams - [Link](#)

Assignments

- ☐ Write the HLD of an SDD for an AI-powered field service management system: architecture diagram, component list, technology stack with rationale and trade-offs, deployment topology (cloud vs on-premise, ERP connection), a model routing policy, and one Architecture Decision Record for your model hosting choice
- ☐ Write the LLD for the same system: API endpoint table (at least 8 endpoints with request/response models), database schema (Equipment, WorkOrder, Technician, SLAPolicy), a sequence diagram for work-order creation, and a second sequence diagram for what happens when the ERP write fails
- ☐ Explain your design out loud in 5 minutes as if in an interview — record yourself, review it, and share the architecture diagram on LinkedIn

Phase 2: Product Management & Soft Skills (Week 13–24)

Phase 1 taught you to build and ship enterprise AI. Phase 2 teaches you to work on the client floor: find the real problem, document it, manage the people, and land the go-live.

Week 17: Problem Discovery & User Research

FDE is like a calm, mature doctor. A patient may come with a complaint of a headache but a calm doctor doesn't give headache medicine immediately. They inquire, test and try to find root cause. Root cause may be you have a gas in your stomach which is causing the headache, so they will find that root cause first and then give medicine. Similarly FDE will find a root cause before building.



Topics

- 5-Whys technique for root cause identification, Moms test
- Interviewing and shadowing end users: mapping their manual process step by step
- User research basics: personas, jobs-to-be-done, buyer (CXO) needs vs end-user needs
- Separating the stated request from the real problem — always ask "what does success look like?"
- Pre-discovery checklist: documents to request — SOPs, org charts, current process metrics
- Discovery interview framework: open-ended questions that uncover real pain
- Facilitating discovery workshops: agenda, room rules, capturing outputs
- Quantifying the problem: error rates, manual hours, SLA breaches, cost per error
- Handling scope creep during discovery; NDA, data governance, and onsite access protocols

Learning Resources

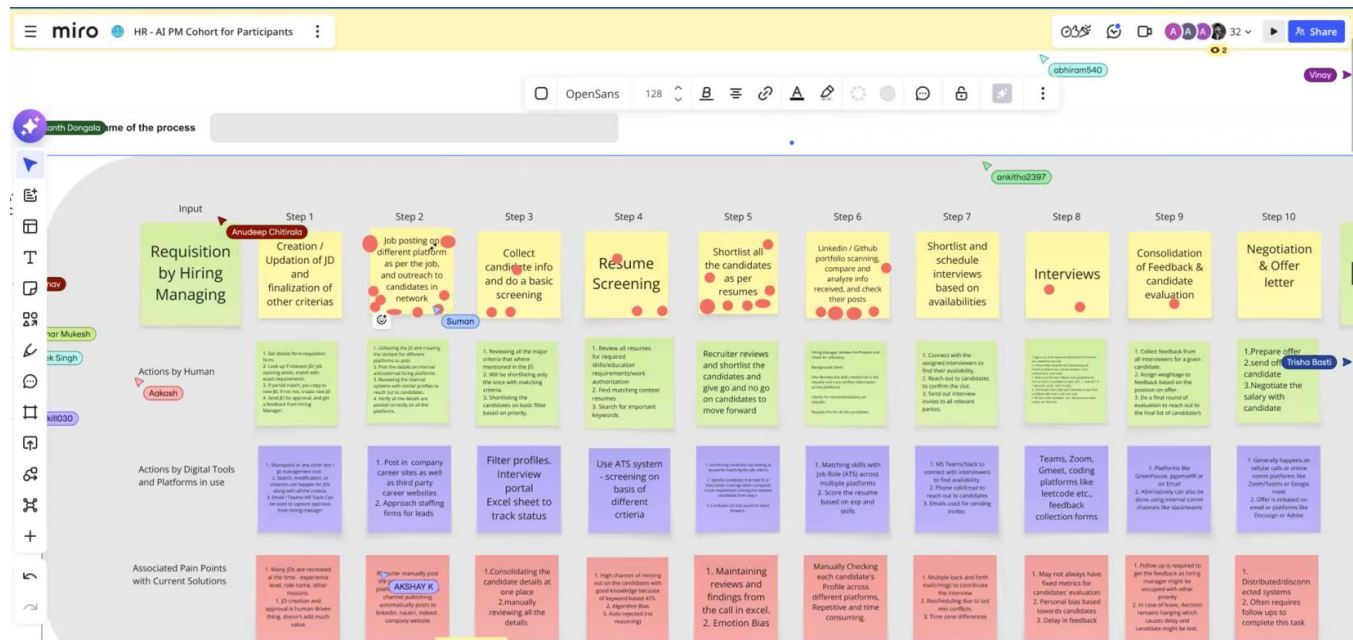
- The Mom Test — how to ask questions customers cannot lie to - [Link](#)
- Customer discovery interview techniques (YouTube) - [Link](#)
- Jobs-to-be-done explained (YouTube) - [Link](#)
- 5 Whys (YouTube) - [Link](#)
- Root cause analysis – [IBM Lecture](#)

Assignments

- ☐ Pick a real process you know (family business, your workplace) and write a discovery interview script with 10 open-ended questions
- ☐ Run the interview with someone who actually does the process — document the AS-IS steps, pain points, and quantify: manual hours, error rate, cost per error
- ☐ Write a one-page problem statement with quantified business impact, and get the interviewee to confirm it is accurate

Week 18: AS-IS Process Mapping with Swim-lane Diagrams

Before automating anything, draw how work happens today. The diagram is how you and the client agree on reality.



Topics

- BPMN (business process model notation) basics: pools, swim-lanes, events, gateways, tasks, connectors
- Drawing **AS-IS** (current state) process flows that capture manual steps precisely
- Identifying automation candidates: repetitive, rule-based, high-volume, error-prone steps
- Defining the **TO-BE** (future AI-automated) vision with stakeholders
- Tools: draw.io (free), Miro (free tier), Eraser.io (AI-native diagrams)

Learning Resources

- BPMN basics with draw.io (YouTube) - [Link](#)
- Swim-lane diagrams in Miro - [Link](#)
- draw.io (free) - [Link](#)

Assignments

- ☐ Draw a complete AS-IS swim-lane diagram for the process you discovered in Week 13–14
- ☐ Highlight in orange the steps you would automate with AI, and note why
- ☐ Share the diagram on LinkedIn with a caption explaining what you learned

Week 19: Business Requirement Document (BRD) Mastery

An enterprise client judges your competence by your documentation before they see a single line of code.

Topics

- BRD structure: Executive Summary, Scope & Out-of-Scope, AS-IS/TO-BE, Functional Requirements, Integration Requirements, Data Requirements, Acceptance Criteria, Risks & Assumptions

- Writing measurable acceptance criteria: not "the system shall be fast" but "work order assignment ≤ 5 seconds at P95"
- Managing scope: what goes out-of-scope and why
- BRD versioning and change log management
- Common mistakes: ambiguous requirements, missing exclusions, untestable criteria

Learning Resources

- BRD & FRD explained — visual & beginner-friendly (YouTube) - [Link](#)
- BRD template walkthrough (YouTube) - [Link](#)
- Mastering BRD documentation (Udemy — paid) - [Link](#)

Assignments

- ☐ Write a full BRD for your Week 15 process — minimum 8 sections, including a functional requirements table with 10+ requirements
- ☐ Get peer feedback: does each requirement have a measurable acceptance criterion?
- ☐ Publish your BRD summary as a LinkedIn carousel post or article

Week 20: Technical Design Document (TDD)

The BRD says what and why. The TDD tells a developer — or an AI coding agent — exactly how the system should behave.

Topics

- TDD vs BRD; deriving functional flows and data flows from the BRD
- Field-level validation specs: mandatory fields, allowed formats, tolerance thresholds
- Business rule specs: e.g. SLA escalation — first warning at 80% of the SLA window, auto-escalation at 100%
- Error scenarios and exception handling requirements
- Integration specs: source systems, APIs, auth, refresh frequency

Learning Resources

- Requirements specification formats (BRD/FRD/SRS) - [Link](#)
- FRD mastery (Udemy — paid) - [Link](#)

Assignments

- ☐ Derive a TDD from your Week 16–17 BRD: functional flows, validation rules, exception handling, integration specs
- ☐ Write one business-rule spec (flowchart + rule table) as if handing it to a developer — then feed it to an AI coding agent and check whether it builds correctly. That is the real test of spec clarity

Week 21, 22: Stakeholder Management & Communication

Technical skills get you shortlisted. These skills decide whether you succeed on the client floor.

Topics

- Stakeholder ecosystem: end-users, process owners, IT gatekeepers, C-suite sponsors — map influence vs interest
- Setting expectations: saying no diplomatically, trading scope for timeline, documenting agreements
- Communication rhythm: meeting agendas, minutes of meeting, weekly status updates, escalation paths
- Presenting to C-suite: the 3-slide format (problem, solution, results)
- Demoing to non-technical audiences: show outcomes, not architecture
- Communication in regional languages: following what the client is saying in Hindi or their local language
- Confidence under uncertainty: clarify with authority — "Help me understand: are you saying X or Y?"

LinkedIn & Brand Building

- Create or audit your LinkedIn profile. Headline: "Aspiring Forward Deployed AI Engineer | Python | LLMs | Enterprise AI"
- Start following: Colin Jarvis (OpenAI), Joe Schmidt (a16z), Dhaval Patel (Codebasics), Andrej Karpathy
- Write 3 LinkedIn posts this month on AI or FDE topics — aim for meaningful engagement
- Help 5 people on LinkedIn or Discord who are asking about FDE topics

Learning Resources

- Toastmasters — join a local chapter - [Link](#)
- How to Avoid Death by PowerPoint (YouTube) - [Link](#)
- "Trading Margin for Moat" — a16z essay on FDEs - [Link](#)
- Business case study deep dives (YouTube — ThinkSchool) - [Link](#)

Assignments

- ☐ Write a 300-word note: "What problem does an FDE solve that a solutions engineer cannot?"
- ☐ Record a 5-minute, 3-slide C-suite presentation of your Phase 1 project (problem, solution, results) and post it on LinkedIn
- ☐ Run a mock stakeholder meeting with a friend playing a difficult process owner — practice scope pushback and capture minutes of meeting

Week 23: UAT & Go-Live

UAT is where the client decides your system works. Go-live is where you prove it.

Topics

- UAT vs QA testing: what UAT validates that unit tests cannot
- Writing UAT test scripts: scenario, pre-conditions, test steps, expected vs actual result, pass/fail
- UAT entry and exit criteria: what must be true before UAT starts and before go-live is approved
- Managing UAT with real end-users: setting expectations, defect triage, severity classification (Critical / High / Medium / Low)
- Go-live checklist: data migration verified, rollback plan documented, support contacts listed
- Hypercare period: escalation matrix, incident response playbook

Learning Resources

- UAT explained (YouTube) - [Link](#)

- Jira fundamentals (Atlassian — free course) - [Link](#)
- Scrum training series (free) - [Link](#)

Assignments

- ☐ Write a 15-scenario UAT test script covering P1 emergency dispatch, SLA breach escalation, parts unavailability, and technician skill mismatch
- ☐ Execute UAT with a friend as the "business user" — track defects found and fix the Priority 1s
- ☐ Write a go-live checklist with a rollback plan for your Phase 1 project

Week 24: Change Management, Handover & Proving ROI

The project is not done when the system works — it is done when the client's team uses it without you.

Topics

- Why end-users resist AI systems and how to address it — the ADKAR model
- End-user training: role-based guides, video walkthroughs, FAQ documents
- KPI design: ROI, efficiency gain, error reduction, SLA compliance %
- Building KPI dashboards a non-technical client can read and trust
- Documentation handover pack: architecture overview, runbook, contact matrix, known issues
- Post-go-live review and continuous improvement feedback loops

Learning Resources

- Prosci ADKAR change management model - [Link](#)
- Change management fundamentals (YouTube) - [Link](#)
- Kanban fundamentals (YouTube) - [Link](#)

Assignments

- ☐ Write a 3-page IT handover document: architecture overview, runbook, contact matrix, known issues
- ☐ Build a KPI dashboard showing: work orders processed, first-time fix rate, SLA compliance %, technician utilisation, MTTR
- ☐ Create a 10-minute end-to-end presentation video of your full 24-week project as if presenting to a client CXO — upload to LinkedIn

Bonus:

Deep SAP Integration & Field Service Domain (Month 7+)

Not required for most FDE roles — but if you target SAP-heavy consultancies or manufacturing / field-service clients, this depth is a differentiator. This section is based on inputs from Pankaj Jaiswal, an FDE working in this exact environment.

SAP Integration Patterns

- SAP integration options: BAPI (Business API), OData (REST-style), RFC (Remote Function Call), IDocs, SAP CPI (Cloud Platform Integration)
- SAP CPI as middleware: integration flows, message mapping, content-based routing
- SAP Datasphere: read-layer views for AI consumption — equipment performance, technician utilization, SLA compliance history
- Writing BAPI integration catalogues: function name, input/output parameters, error codes (e.g. BAPI_ALM_ORDER_MAINTAIN for work order updates)

SAP Domain Knowledge — Manufacturing / Field Service

- SAP PM (Plant Maintenance): equipment master (IE01/IE03), functional locations (IL01), work orders (IW31/IW32/IW33), notifications (IW51)
- SAP CS (Customer Service): service orders, service contracts, resource-related billing
- Spare parts management: material reservations (MB21), goods issue (MB1A), stock overview (MMBE)
- SLA management: response time vs resolution time, P1–P4 priority matrix, penalty clauses
- Field technician scheduling: skill-based routing, geography-based dispatch, availability calendars
- IoT integration basics: how sensor data (vibration, temperature, pressure) triggers predictive maintenance work orders
- Warranty management and Indian manufacturing compliance: BIS certifications, statutory inspection logs, safety audit trails

Learning Resources

- SAP OData introduction (SAPYard) - [Link](#)
- SAP Community — BAPI to OData guide - [Link](#)
- SAP PM module overview & IW31 work orders (YouTube) - [Link](#)
- Field Service SLA concepts + IoT predictive maintenance (YouTube) - [Link](#)

Assignment

- ☐ Write a BAPI Integration Catalogue table with 5 BAPIs for field service automation (create work order, read equipment master, confirm work order operation, create service notification, check material availability)

The 12 Principles of a Great Forward Deployed Engineer

Technical skills get you shortlisted. These principles determine whether you succeed or fail on the client floor.

PRINCIPLE 1 — Communication

Good communication is not about speaking English fluently. If your client's team primarily speaks Hindi, you need to understand and respond in Hindi. Many enterprise clients in India still operate in regional languages internally. An FDE who cannot follow what the client is actually saying in their natural language will miss the real requirement.

PRINCIPLE 2 — Adaptability

Every client has a different culture, tech stack, hierarchy, and pace. What worked at one client may fail at another. An FDE must adapt quickly — to the environment, to the tools available, to the communication style of each stakeholder, and to changing requirements without losing momentum.

PRINCIPLE 3 — AI-Native Work Style

Be fully comfortable using AI tools (Claude, ChatGPT, Gemini, Cursor, Eraser.io) to accelerate your work without compromising output quality. AI-native FDEs are 3x faster than those who resist AI tools. The bar is not "did you use AI" — it is "did you deliver quality output on time."

PRINCIPLE 4 — Confidence Under Uncertainty

In requirements gathering, if you do not understand something, never show underconfidence. Ask a clarifying question with authority: "Help me understand this better — are you saying X or Y?" Use LLMs to quickly understand unfamiliar concepts before your next client meeting. The client must always trust your competence.

PRINCIPLE 5 — Speed

Enterprise timelines are unforgiving. Clients have board deadlines, audit cycles, and seasonal cutoffs. The ability to move fast — from a workshop to a working prototype in 48 hours — is what separates great FDEs from good ones. Shipping a rough working demo beats a polished presentation every time.

PRINCIPLE 6 — Cost-Efficient AI, Not Just Functional AI

Take a bottom-up approach to LLM selection: use lightweight models (Flash, Haiku, Mistral) for simple classification tasks and reserve GPT-4o or Sonnet only where accuracy demands it. Many startups are solving the same enterprise problems — cost efficiency is your competitive moat.

PRINCIPLE 7 — Build Agentic, Not Hardcoded

Enterprise clients send change requests constantly. If your application is hardcoded, every change requires a developer. Build agentic systems where business logic can be updated by the client team themselves — through natural language instructions, configurable rules, or no-code interfaces.

PRINCIPLE 8 — Always Include a Memory Layer

Once the AI development team moves to another project, they cannot be on call forever. Enterprise AI applications must have a memory layer so the agent auto-learns from corrections and user feedback over time — without requiring developer intervention for every accuracy issue.

PRINCIPLE 9 — Know On-Premise Deployment

"How does this deploy on our servers?" is one of the most frequently asked questions at enterprise clients. Learn how on-premise Kubernetes (kubeadm, Rancher), air-gapped Docker registries, and private LLM hosting (Ollama, vLLM) work. Many large enterprises in India and regulated industries cannot move data to public cloud.

PRINCIPLE 10 — AI Governance Awareness

AI Governance is no longer optional — it is becoming a compliance requirement. Understand the basics: data privacy (DPDP Act, GDPR), model explainability, audit trails, bias monitoring, and responsible AI frameworks. FDEs who understand this speak the language of the client's legal and compliance teams.

PRINCIPLE 11 — Know the ERP Landscape

As an FDE, you will almost always be building AI on top of an ERP system — SAP, Oracle NetSuite, Microsoft Dynamics, Tally, or a custom ERP. You do not need consultant-level depth: in practice, FDEs fetch ERP data through pre-built connectors. But you must know the landscape — what the system of record is, which

module owns which data, and how data gets out (connectors, OData/REST APIs) — or you cannot design the right integration. Go deeper only if you specialize in SAP-heavy clients.

PRINCIPLE 12 — User Training for a Broad Audience

The end-users of your enterprise AI application are often non-technical — HR managers, procurement officers, finance executives. Making them understand how the AI works, what it does, where to intervene, and who to call when something goes wrong is as important as building the system. Great FDEs run training sessions and leave behind simple, visual user guides.

AI Tools to Accelerate Your Delivery Workflow

An FDE who uses AI tools intelligently produces better deliverables in half the time. These are the tools actively used on real enterprise projects — not just in theory.

#	Tool	Best Used For	Pro Tip
1	Claude (Anthropic)	Generating BRDs, FRDs, System Design Documents, meeting summaries, email drafts, validation rule tables	Use structured XML prompts with clear section templates. Claude maintains long document context better than most models — ideal for iterating on multi-page enterprise docs.
2	ChatGPT (OpenAI)	Generating system architecture diagrams, flowcharts, and data flow visuals using its image generation capability	Describe your components and connection lines clearly. ChatGPT image generation works well for producing clean architecture diagrams from a text description.
3	Multi-Model Approach	When using LLMs for repetitive tasks — especially image or diagram generation — and you see consistent errors, switch models	If Claude gets a diagram wrong repeatedly, try ChatGPT. If ChatGPT keeps hallucinating a data flow, try Gemini. Do not keep retrying the same model for the same structural mistake.
4	Eraser.io	Generating system design diagrams, flow diagrams, and sequence diagrams using AI-assisted DSL (diagram-as-code)	Use Eraser's DSL with direction right for landscape swimlane layouts. Describe actors, systems, and data flows in plain English and Eraser converts it to a clean diagram.
5	Cleusco	Recording walkthrough and demo videos for client training, UAT handholding, and stakeholder presentations	Record your screen with Cleusco to produce polished walkthrough videos for end-user training. A 3-minute video explaining how to use a feature replaces 10 pages of written user guide for non-technical users.

Real-World Case Studies

#	Case Study	Key Technical Elements	Documents Produced
01	AI Field Service Management Command Center Multi-agent system for automated work order triage, skill-based technician dispatch, real-time SLA breach prediction,	LangGraph Supervisor, GPT-4o, SAP PM OData, Azure SQL, Datasphere, IoT event stream	BRD, FRD, SDD with agent diagrams, SAP BAPI Integration Catalogue, UAT scripts

#	Case Study	Key Technical Elements	Documents Produced
	and spare parts availability check.		
02	Predictive Maintenance AI Agent Agentic system that ingests IoT sensor data, detects anomaly patterns, auto-creates SAP PM work orders before equipment failure, and routes to the right technician.	LangGraph, Gemini 2.5 Flash, SAP PM BAPI, IoT Hub, FastAPI, LangSmith	SDD with IoT integration spec, BAPI catalogue, anomaly detection rule library, KPI dashboard design
03	Contract Lifecycle Management AI Agent AI agent that extracts key clauses from service contracts, flags SLA breaches and renewal deadlines, and answers natural language questions about contract terms.	Claude Sonnet, CrewAI, pgvector, Python FastAPI, MS Graph API, RAGAS evaluation	Prompt library, clause extraction templates, evaluation report, handover runbook